A New AML Era

David L. Woodruff
Graduate School of Management
University of California, Davis
DLWoodruff@UCDavis.edu

Abstract

Claude Code translated a published unit commitment formulation directly into a C program that calls Gurobi, with no algebraic modeling language in between. On the pglib-uc instances the program agrees with the authors' reference implementation, except where the program follows the paper more closely, and Claude Code found an error in one of the published equations. Independent verification is still needed, and we discuss its limits.

Algebraic modeling languages (AMLs) let a modeler state an optimization problem in something close to the indexed algebraic notation of a textbook. An AML serves as a translator to produce the data structures a solver needs. This note asks the question: Why not let a large language model (LLM) go directly from the algebraic notation to an implementation in, say, the C programming language and "cut out the middleman" so to speak? We suggest a few answers to this question, but also demonstrate using Claude Code [4] that it is a very reasonable question.

1 Literature Review Mainly Done by Claude Code

Early steps toward machine-readable symbolic statements of mathematical programs include Pugh's language for nonlinear programming [72] and the UIMP interface [25], and Fourer made the systematic case for modeling languages over the matrix generators then in wide use [26]. GAMS, which began at the World Bank, is generally regarded as the first AML to see broad use [9, 11, 12, 66], and AMPL followed [30–32]. AMPL was later extended toward combinatorial optimization and constraint programming [27, 29]. Other long-lived systems include LINGO from LINDO Systems [19], AIMMS [10], LPL [46, 47], MPL from Maximal Software [57], Mosel from the Xpress family of products [17], and OPL, which brought together mathematical and constraint programming [84, 85], along with the MATLAB-based TOMLAB environment [44] and SAMPL, an extension for stochastic programming [83]. Most of these systems are or became commercial products, but not all of the early languages did: CAMPS [64], MODLER [39], the MINOPT language [77], and ASCEND [71], which was later released under the GPL. Another early system is PAM [87], and related conceptual work includes Geoffrion's structured modeling [35] and Bisschop and Kuip's treatment of compound and hierarchical sets [7, 8]. Surveys and comparisons of this generation of systems are available in [28, 40, 51, 52, 58, 74].

Open-source AMLs have since multiplied, and most of the recent ones are embedded in a general-purpose host language rather than defined as stand-alone languages. We limit this overview to open-source systems that have a citable publication (a paper, report, or thesis), so we omit tools documented mainly through a manual or repository, such as GNU MathProg, as well as tools that are free of charge but not distributed under an open-source license, such as YALMIP [60, 61]

and ROME [36]. We also read "AML" broadly enough to include closely related modeling frameworks. Stand-alone languages include ZIMPL [21,56], CMPL [80], the structure-conveying SML [18], the graph-based GBOML [67], and, for constraint programming, MiniZinc [69] and Essence [33]. C++ hosts FlopC++ [45, 88] and Gravity [42]. In Python, Pyomo [13, 41] and its extensions for stochastic programming [53, 86], differential-algebraic models [70], and generalized disjunctive programming [14] sit alongside PuLP [68], PySCIPOpt [65], optlang [50], GEKKO [5], PICOS [75], Linopy [43], RSOME [15, 16], and PyOptInterface [89]. In Julia, JuMP [23, 62, 63] and its MathOptInterface layer [59] anchor an ecosystem that includes SDDP.jl [22], StochasticPrograms.jl [6], InfiniteOpt.jl [73], Plasmo.jl [49], and ExaModels.jl [79]. A distinct line of work builds on disciplined convex programming [37,38]: CVX in MATLAB, CVXPY in Python [1,20] with its DCCP [78] and CVXPYgen [76] extensions, Convex.jl in Julia [82], and CVXR in R [34], where the ROI infrastructure [81] also provides a modeling interface. For dynamic optimization, CasADi [3] and Optimica with JModelica.org [2] provide symbolic and language-based alternatives.

2 The Experiment

Our test problem is the unit commitment model of Knueven, Ostrowski, and Watson [55]. Claude Code worked from the version of that paper posted on Optimization Online [54], not from the journal version, and all equation numbers below refer to the Optimization Online version. A later check by Claude Code confirmed that the equations of formulation (T) carry the same numbers in the journal version. We use the "Tight" formulation, denoted (T), which those authors identify as the best performer among the formulations they tested. Appendix A reproduces it.

We used Claude Code version 2.1.318 with the model Claude Opus 5.5 (`claude-opus-5-5`) in November 2026. The program was built with GCC 15.2.0 and CMake 4.2.3, linked against Gurobi 13.0.2, and run on Ubuntu 26.04.1. The comparison runs used version 0.6.2 of the open-source EGRET package [24], which implements (T), with Pyomo 6.9.5.dev0, gurobipy 12.0.2, and Python 3.12.16, and the instances came from pglib-uc [48] at commit `39a7f38`. All of the runs were done on an Acer Swift laptop (model SFX16-61G) with an AMD Ryzen 7 7840HS processor (8 cores, 16 threads) and about 14 GB of usable memory.

2.1 Creation of the C Code

A Claude Code session received the Optimization Online PDF, seven pglib-uc instances spanning the three benchmark families, a description of the data format, and a prompt asking for (T) as a C program that passes the model to Gurobi through its C API, with no AML. The prompt supplied C_{LP}, C_{RP}, and C^w, which the data omit, and told the session to work only from the paper and the data. It ran in a sandbox containing no implementation of the model, with web tools denied by permission rules, and its log shows no use of another implementation.

Claude Code wrote the program, including a JSON reader, and a CMake file, with no human intervention. It reached a first program that built and solved in about 13 minutes, before any testing, and finished in 57 minutes, which includes solving all seven instances, measuring the effect of its decisions, and writing up its results and notes.

Working from the paper alone, Claude Code found that (T) cannot be implemented exactly as published, and it recorded each departure with its reason; Appendix B lists them. Two of them matter numerically. Taken literally, the ranges in (4) and (5) are empty for generators whose minimum times exceed the horizon, which it measured as lowering the cost of one FERC instance by 12.7%, and (67) as published raises the cost of one RTS-GMLC instance by 135%. It flagged its change to (67) as the decision we might want to overrule, and we kept it.

None of these troubles with (T) are in EGRET 0.6.2 [24], as we checked afterward: EGRET has the corrected $K^{SU}(t)$, additional term in (41), and (48), includes $p_W(t)$ in (67), and caps minimum times at T so that (4) and (5) are not empty.

2.2 Validation Using EGRET and Refinement

We then compared the program with formulation (T) in EGRET. Six of the seven LP bounds agree to six decimal places, and on all seven instances the intervals from MIP bound to objective overlap, at a 0.01% gap. A separate checker recomputed the cost of each solution and tested it against the operating constraints in the data. Any failed item would start a new session to refine the program, showing it these numbers but neither EGRET nor the checker.

The checker raised one flag, a reserve shortfall of 0.000011 MW on one instance, and that started a refinement session of 37 minutes. The flag was a false alarm from the checker's tolerance of 10^{-5}: solutions are written with six decimals, so rounding across the 978 generators of that instance can reach about 10^{-3}. The session reached this conclusion and changed nothing in the model. We did not change the tolerance and repeat the check, so the recorded result still shows this flag.

Without seeing EGRET's code, and given only EGRET's numbers, the same session traced the remaining LP difference, on one RTS-GMLC instance, to (35) at $t = 1$. The session correctly concluded that the program follows the paper and that EGRET uses a weaker form of that row, one that gives a lower LP bound but has the same integer solutions. The session guessed that form slightly wrong: EGRET's code adds $\underline{P}w(1)$ to the right-hand side, not the inferred $RU\,w(1)$. Both forms give EGRET's LP bound on that instance. Section 2.3 shows that only EGRET's form matches across all of pglib-uc.

So no refinement of the model was needed. That does not make verification optional, for a translator of any kind. A person copying (4) and (5) into a conventional AML would get the same empty ranges and the same clean solve, and a comparison with EGRET would expose them. But comparing numbers also has limits. Equation (48) as published is invalid, as Claude Code found by reading it, yet the published and corrected forms coincide when $SU = SD$, and no generator with $UT = 1$ in these seven instances has $SU \neq SD$. Solving all seven with the published form gave exactly the same LP bounds, MIP objectives, and MIP bounds, so no comparison on these instances could reveal the error. Verification can also fail in the other direction, as the false alarm shows.

The checks here are also not fully independent. A separate Claude Code session, which had already read EGRET and seen the data, wrote the prompts, the data description, and the checker. The two checking tools that the translating session wrote for itself compile but have never been run, because the permissions we set blocked them.

2.3 All of pglib-uc

We then solved the LP relaxations of all 56 instances in pglib-uc with the same program, unchanged; 49 of these instances had played no part in building or checking it. The reference for this comparison is EGRET 0.6.2 with a pull request applied, which has since been merged.[1] Of the 56 LP bounds, 43 agree, with a largest difference of 8×10^{-6} on an objective near 10^8. On the other 13 the program's bound is higher, by at most 0.008%, and all of these come from the difference in (35) at $t = 1$ described above: the program's row is valid and tighter, and a copy of the program using EGRET's

[1] `https://github.com/grid-parity-exchange/Egret/pull/342`, merged into EGRET's main branch as commit `c2a3c63`.

row matches on all 56. A Claude Code session that could read EGRET's code made this comparison and the copy.

This check covers LP bounds only, so it does not test integer solutions. We also solved the MIP on those 13 instances, and the intervals from MIP bound to objective overlap. The original experiment used EGRET 0.6.2; rerunning against the version with this pull request gave the same LP bounds and overlapping MIP intervals on the seven instances.

3 Discussion

We aren't really cutting out the AML "middleman" tasks; we are decomposing them. The interpretation of the algebraic model is done by an LLM and the instantiation of the model in a solver is done by C code. It took the LLM about 13 minutes to read the entire paper and then produce the C program for one of the models in it.

An AML and an LLM are not interchangeable translators, however. An AML translates the same way every time and leaves behind a readable model that can be maintained and passed to other solvers, whereas the LLM left us more than a thousand lines of C tied to one solver's API (on the other hand, having the LLM rewrite for a new solver is easy). The LLM also did more than translate: working from the paper alone, it found five problems in the published formulation and resolved them with its own judgment, including a change to (67) that moves the cost of one instance by 135%. That ability is useful, but it is also why the result needs independent verification, and any regenerated program would need checking again.

One can imagine workflows where models are developed using algebraic notation which is given to an LLM so that it can produce a compiled program for use in production. Changes to the model would follow a similar workflow. But one can also imagine workflows where models are developed in an AML and the LLM can then produce a compiled program for use in production. It would seem to be mainly a matter of preference by the modeler. One can imagine a preference for an AML embedded in a programming language when the model specification itself includes procedural definitions in addition to algebraic notation. However, an LLM can also process procedural definitions expressed in a variety of ways.

At the time of this writing, the use of LLMs in optimization model development is still evolving. Certainly, LLMs are very good at translating models from one format to another, freeing modelers to express their models however they like and rely on an LLM to translate them fairly quickly.

Code and Data Availability

The program, the prompts, the experiment scripts, the instances, and the records of the sessions, with the paper's content removed from those records, are available at `https://github.com/DLWoodruff/new-aml-era`.

Acknowledgement

Claude Code drafted much of the text of this note, including the literature review, and wrote the program and the checks. The author directed the work, edited the text, and is responsible for the final content.

A The Tight Formulation of Knueven, Ostrowski, and Watson

Apart from a few linking remarks, everything in Sections A.1 and A.2, including the notation and the equation numbers, is due to Knueven, Ostrowski, and Watson and is reproduced from Table 3 and Section 3 of the Optimization Online version of their paper [54] (published as [55]). They assembled (T) from their own components and from components of earlier authors, whom they credit in their paper. Claude Code transcribed the equations from the source PDF, and any transcription errors are not the authors'. All equations appear as published, including those the program implements differently; remarks on the implementation are confined to Appendix B.

A.1 Notation

As in the source, the generator subscript g is dropped where it is clear, constraints hold for each $g \in \mathcal{G}$ and $t \in \mathcal{T} = \{1, \dots, T\}$ unless stated otherwise, $v(t)$ and $w(t)$ for $t < 1$ come from historical data, and $v(t)$ and $w(t)$ for $t > T$ are 0.

Parameters. $\underline{P}$, $\overline{P}$: minimum and maximum power output. $\overline{P}^l$: maximum power available for piecewise segment $l \in \mathcal{L}$, with $\overline{P}^0 = \underline{P}$. RU, RD, SU, SD: ramp-up, ramp-down, start-up, and shut-down rates. UT, DT: minimum up time and down time. U, D: hours the generator must be on or off at $t = 1$. C^l: cost coefficient of segment l. C^R: cost of running at $\underline{P}$. C^s: start-up cost in category $s \in \mathcal{S} = \{1, \dots, S\}$, from hottest to coldest. $\underline{T}^s$: time offline after which category s is used, with $\underline{T}^1 = DT$ and $\underline{T}^S = TC$. C^w: shut-down cost. C_{LP}, C_{RP}: penalties on load mismatch and reserve shortfall. $D_n(t)$: load at bus n. $R(t)$: spinning reserve requirement. $\underline{W}_n(t)$, $\overline{W}_n(t)$: must-take and available renewable generation at bus n. B_k, F_k: susceptance and capacity of branch k, which runs from bus $k(n)$ to bus $k(m)$.

Variables. $u(t), v(t), w(t) \in \{0, 1\}$: on, start-up, and shut-down status. $p'(t) \geq 0$: power above minimum, with $p(t) = p'(t) + \underline{P}u(t)$ (12). $\overline{p}'(t) \geq 0$: maximum power available above minimum, with $\overline{p}(t) = \overline{p}'(t) + \underline{P}u(t)$ (13). $p^l(t) \geq 0$: power from segment l. $c^p(t), c^{SU}(t), c^{SD}(t)$: production cost above $\underline{P}$, start-up cost, and shut-down cost. $x(t, t') \in \{0, 1\}$: shut-down at t and start-up at t', for $t' - TC < t \leq t' - DT$. $p_{W,n}(t) \geq 0$: renewable generation used at bus n. $f_k(t)$: flow on branch k. $\theta_n(t)$: voltage angle at bus n. $s_n(t)$, $s_n^+(t), s_n^-(t) \geq 0$: load mismatch and its positive and negative parts. $s_R(t) \geq 0$: reserve shortfall.

Derived quantities.

$$T^{RU} = \left\lfloor \frac{\overline{P} - SU}{RU} \right\rfloor, \qquad T^{RD} = \left\lfloor \frac{\overline{P} - SD}{RD} \right\rfloor, \qquad K^{SD}(t) = \min\{T^{RD},\, UT - 1,\, T - t - 1\},$$

$$K^{SU}(t) = \min\{T^{RU},\, UT - 2 - [K^{SU}(t)]^+,\, t - 1\}, \qquad [\,\cdot\,]^+ := \max\{\cdot, 0\}.$$

$$C^v(l) := \begin{cases} 0 & \overline{P}^l \leq SU \\ \overline{P}^l - SU & \overline{P}^{l-1} < SU < \overline{P}^l \\ \overline{P}^l - \overline{P}^{l-1} & \overline{P}^{l-1} \geq SU \end{cases} \qquad C^w(l) := \begin{cases} 0 & \overline{P}^l \leq SD \\ \overline{P}^l - SD & \overline{P}^{l-1} < SD < \overline{P}^l \\ \overline{P}^l - \overline{P}^{l-1} & \overline{P}^{l-1} \geq SD \end{cases}$$

A.2 The formulation

Objective.

$$\min \sum_{t\in\mathcal{T}} \left(\sum_{g\in\mathcal{G}} \left(c_g^p(t) + C_g^R u_g(t) + c_g^{SU}(t) + c_g^{SD}(t)\right) + \sum_{n\in\mathcal{N}} C_{LP}\left(s_n^+(t) + s_n^-(t)\right) + C_{RP}\, s_R(t) \right). \tag{69}$$

The cost variables defined by equalities below are substituted directly into (69).

Linking relations.

$$p(t) = p'(t) + \underline{P}u(t) \tag{12}$$
$$\overline{p}(t) = \overline{p}'(t) + \underline{P}u(t) \tag{13}$$

Up-time and down-time.

$$u(t) - u(t-1) = v(t) - w(t) \qquad t \in \mathcal{T} \tag{2}$$
$$\sum_{i=1}^{\min\{U,T\}} u(i) = \min\{U,T\} \tag{3a}$$
$$\sum_{i=1}^{\min\{D,T\}} u(i) = 0 \tag{3b}$$
$$\sum_{i=t-UT+1}^{t} v(i) \le u(t) \qquad t \in \{UT,\dots,T\} \tag{4}$$
$$\sum_{i=t-DT+1}^{t} w(i) \le 1 - u(t) \qquad t \in \{DT,\dots,T\} \tag{5}$$

Generation limits. Constraint (17) holds for all generators, and (23) holds for generators with $UT = 1$:

$$p'(t) \le \overline{p}'(t) \tag{17}$$
$$\overline{p}'(t) \le (\overline{P} - \underline{P})u(t) - (\overline{P} - SU)v(t) - [SU - SD]^+ w(t+1) \tag{23a}$$
$$\overline{p}'(t) \le (\overline{P} - \underline{P})u(t) - (\overline{P} - SD)w(t+1) - [SD - SU]^+ v(t) \tag{23b}$$

The source states (23) with $p'(t) + r(t)$ on the left, which equals $\overline{p}'(t)$ by its equation (14). The variable upper bounds are (38), then (40) if $T^{RU} > UT - 2$, and (41) if $K^{SD}(t) > 0$:

$$\overline{p}(t) \le \overline{P}u(t) - (\overline{P} - SD)w(t+1) - \sum_{i=0}^{\min\{UT-2,\, T^{RU}\}} (\overline{P} - SU - iRU)\, v(t-i) \tag{38}$$
$$\overline{p}(t) \le \overline{P}u(t) - \sum_{i=0}^{\min\{UT-1,\, T^{RU}\}} (\overline{P} - SU - iRU)\, v(t-i) \tag{40}$$
$$p(t) \le \overline{P}u(t) - \sum_{i=0}^{K^{SD}(t)} (\overline{P} - (SD + iRD))\, w(t+1+i) - \sum_{i=0}^{K^{SU}(t)} (\overline{P} - (SU + iRU))\, v(t-i) \tag{41}$$

If $K^{SU}(t) < T^{RU}$, the term $-[(SU + (K^{SU}(t)+1)RU) - (SD + T^{RD}RD)]^+ v(t - (K^{SU}(t)+1))$ is added to the right-hand side of (41).

Ramp limits.

$$\overline{p}'(t) - p'(t-1) \leq (SU - \underline{P} - RU)v(t) + RU\, u(t) \tag{35}$$
$$p'(t-1) - p'(t) \leq (SD - \underline{P} - RD)w(t) + RD\, u(t-1) \tag{36}$$

Piecewise production. Constraints (43) and (44) hold for all generators, (46) holds if $UT > 1$, and (48) holds otherwise:

$$\sum_{l\in\mathcal{L}} p^l(t) = p'(t) \tag{43}$$
$$\sum_{l\in\mathcal{L}} C^l p^l(t) = c^p(t) \tag{44}$$
$$p^l(t) \leq (\overline{P}^l - \overline{P}^{l-1})u(t) - C^v(l)v(t) - C^w(l)w(t+1) \qquad l \in \mathcal{L} \tag{46}$$
$$p^l(t) \leq (\overline{P}^l - \overline{P}^{l-1})u(t) - C^v(l)v(t) - [C^v(l) - C^w(l)]^+ w(t+1) \qquad l \in \mathcal{L} \tag{48a}$$
$$p^l(t) \leq (\overline{P}^l - \overline{P}^{l-1})u(t) - C^w(l)w(t+1) - [C^w(l) - C^v(l)]^+ v(t) \qquad l \in \mathcal{L} \tag{48b}$$

Start-up and shut-down cost.

$$\sum_{t'=t-TC+1}^{t-DT} x(t',t) \leq v(t) \tag{59}$$
$$\sum_{t'=t+DT}^{t+TC-1} x(t,t') \leq w(t) \tag{60}$$
$$c^{SU}(t) = C^S v(t) - \sum_{s=1}^{S-1} (C^S - C^s) \left(\sum_{t'=t-\underline{T}^{s+1}+1}^{t-\underline{T}^s} x(t',t) \right) \tag{61}$$
$$c^{SD}(t) = C^w w(t) \tag{62}$$

System constraints.

$$\underline{W}_n(t) \leq p_{W,n}(t) \leq \overline{W}_n(t) \qquad \forall t \in \mathcal{T},\ \forall n \in \mathcal{N} \tag{63}$$
$$f_k(t) = B_k \left(\theta_{k(n)}(t) - \theta_{k(m)}(t)\right) \qquad \forall k \in \mathcal{K},\ \forall t \in \mathcal{T} \tag{64a}$$
$$-F_k \leq f_k(t) \leq F_k \qquad \forall k \in \mathcal{K},\ \forall t \in \mathcal{T} \tag{64b}$$
$$-\pi \leq \theta_n(t) \leq \pi \qquad \forall n \in \mathcal{N},\ \forall t \in \mathcal{T} \tag{64c}$$
$$\theta_1(t) = 0 \qquad \forall t \in \mathcal{T} \tag{64d}$$
$$\sum_{g\in\mathcal{G}_n} p_g(t) + p_{W,n}(t) + \sum_{k\in\delta^+(n)} f_k(t) - \sum_{k\in\delta^-(n)} f_k(t) + s_n(t) = D_n(t) \qquad \forall n \in \mathcal{N},\ \forall t \in \mathcal{T} \tag{65}$$
$$s_n(t) = s_n^+(t) - s_n^-(t) \qquad \forall n \in \mathcal{N},\ \forall t \in \mathcal{T} \tag{66a}$$
$$s_n^+(t),\ s_n^-(t) \geq 0 \qquad \forall n \in \mathcal{N},\ \forall t \in \mathcal{T} \tag{66b}$$
$$\sum_{g\in\mathcal{G}} \overline{p}_g(t) + s_R(t) \geq \sum_{n\in\mathcal{N}} D_n(t) + R(t) \qquad \forall t \in \mathcal{T} \tag{67}$$

Here $\delta^+(n)$ and $\delta^-(n)$ are the lines to and from bus n. With only one bus, the authors drop (64), and the flow sums in (65) are empty, which leaves a system-wide demand constraint.

B Where the Program Departs from the Published Formulation

Claude Code identified and implemented these departures without seeing EGRET, as Section 2.1 describes, and we compared them with EGRET 0.6.2 [24] only afterward. The first five items correct the published formulation. The published forms appear the same way in the journal version [55], so "published" below refers to both versions, and EGRET 0.6.2 is correct on all five. The additional term in (41) was corrected in EGRET in a 2023 change first included in release 0.6.0, whereas $K^{SU}(t)$ and (67) have had the forms described here since EGRET's first public commit in April 2019.

- **The definition of $K^{SU}(t)$.** As published, this definition refers to itself. The program uses $K^{SU}(t) = \min\{T^{RU},\, UT - 2 - [K^{SD}(t)]^+,\, t-1\}$, which is what EGRET computes.
- **The additional term in (41).** The program uses $-[(SD + K^{SD}(t)RD) - (SU + (K^{SU}(t) + 1)RU)]^+$ as the coefficient of $v(t-(K^{SU}(t)+1))$, with the same difference and sign as current EGRET. It adds the term whenever $K^{SU}(t) < T^{RU}$, as the source says, whereas EGRET adds it in fewer cases. The published coefficient can cut off feasible points. For example, take $\underline{P} = 0$, $\overline{P} = 100$, $SU = 59$, $RU = 20$, $SD = 10$, $RD = 20$, and $UT = 7$, so that $T^{RU} = 2$, $T^{RD} = 4$, $K^{SD}(t) = 4$, and $K^{SU}(t) = 1$ away from the ends of the horizon. A generator that starts at $t-2$ and stays on may produce 99 at t, but the published term gives $p(t) \le 91$. The corrected term left the LP bound unchanged on the three instances where this was measured.
- **Equation (48).** The operands of $[\cdot]^+$ are reversed as published, and the program uses $[C^w(l) - C^v(l)]^+$ in (48a) and $[C^v(l) - C^w(l)]^+$ in (48b), as EGRET does. The published form is invalid. With $\underline{P} = 10$, breakpoints $10, 30, 50$, $SU = 20$, $SD = 40$, and $UT = 1$, published (48a) gives $p^2(t) \le -10$ for a one-period run, which is feasible. The two forms coincide when $SU = SD$.
- **Reserves.** The program adds $p_W(t)$ to the left-hand side of (67), as EGRET does. The reserve requirement in the source's (68) is equivalent to the version with this term when load is met exactly, using (65) and $r(t) = \overline{p}(t) - p(t)$, whereas (67) as published in effect requires thermal generators to cover the renewable output as well.
- **Long minimum times.** Some instances have UT or DT greater than T, in which case the ranges in (4) and (5) are empty and the minimum time would not be enforced. The program uses the ranges $\{\min\{UT, T\}, \ldots, T\}$ and $\{\min\{DT, T\}, \ldots, T\}$, with the sums clipped to the horizon. EGRET also caps these minimum times at T.
- **Network.** The pglib-uc instances have no network, so the program uses the one-bus case of Knueven, Ostrowski, and Watson and a single aggregate renewable variable per period.
- **History.** The program takes $v(1 - \mathit{time_up_t0}) = 1$ for a generator on at time 0 and $w(1 - \mathit{time_down_t0}) = 1$ for one that is off, and sets other historical values to 0. For a generator that is off at time 0, the only arcs $x(t', t)$ with $t' < 1$ are those from that shut-down.
- **Data handling.** The program clips SU and SD at $\overline{P}$, because most generators in the CAISO instances have limits above it, and it sets $u(t) = 1$ for generators flagged `must_run` in the data. The pglib-uc files carry no penalties or shut-down costs, so as the prompt specified it uses $C_{LP} = 10000$, $C_{RP} = 1000$, and $C^w = 0$.

The program follows the published (35) at $t = 1$, which differs from EGRET. EGRET's right-hand side is larger by $\underline{P}w(1)$, which accounts for the one LP difference in Section 2.2. Both forms are valid and have the same integer solutions, and the published one is tighter.

References

[1] Akshay Agrawal, Robin Verschueren, Steven Diamond, and Stephen Boyd. A rewriting system for convex optimization problems. *Journal of Control and Decision*, 5(1):42–60, 2018.

[2] J. Åkesson, K.-E. Årzén, M. Gäfvert, T. Bergdahl, and H. Tummescheit. Modeling and optimization with Optimica and JModelica.org—Languages and tools for solving large-scale dynamic optimization problems. *Computers & Chemical Engineering*, 34(11):1737–1749, 2010.

[3] Joel A. E. Andersson, Joris Gillis, Greg Horn, James B. Rawlings, and Moritz Diehl. CasADi: a software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1):1–36, 2019.

[4] Anthropic. Claude Code. Version 2.1.318, with model Claude Opus 5.5, `https://claude.com/claude-code`, 2026.

[5] Logan D. R. Beal, Daniel C. Hill, R. Abraham Martin, and John D. Hedengren. GEKKO Optimization Suite. *Processes*, 6(8):106, 2018.

[6] Martin Biel and Mikael Johansson. Efficient Stochastic Programming in Julia. *INFORMS Journal on Computing*, 34(4):1885–1902, 2022.

[7] J. J. Bisschop and C. A. C. Kuip. Compound Sets in Mathematical Programming Modeling Languages. *Management Science*, 39(6):746–756, 1993.

[8] J. J. Bisschop and C. A. C. Kuip. Hierarchical sets in mathematical programming modeling languages. *Computational Optimization and Applications*, 1(4):415–438, 1993.

[9] Johannes Bisschop and Alexander Meeraus. On the development of a general algebraic modeling system in a strategic planning environment. *Mathematical Programming Study*, 20:1–29, 1982.

[10] Johannes Bisschop and Marcel Roelofs. The Modeling Language AIMMS. In Josef Kallrath, editor, *Modeling Languages in Mathematical Optimization*, volume 88 of *Applied Optimization*, pages 71–104. Kluwer Academic Publishers, 2004.

[11] Anthony Brooke, David Kendrick, and Alexander Meeraus. *GAMS: A User's Guide*. The Scientific Press, 1988.

[12] Michael R. Bussieck and Alex Meeraus. General Algebraic Modeling System (GAMS). In Josef Kallrath, editor, *Modeling Languages in Mathematical Optimization*, volume 88 of *Applied Optimization*, pages 137–157. Kluwer Academic Publishers, 2004.

[13] Michael L. Bynum, Gabriel A. Hackebeil, William E. Hart, Carl D. Laird, Bethany L. Nicholson, John D. Siirola, Jean-Paul Watson, and David L. Woodruff. *Pyomo—Optimization Modeling in Python*, volume 67 of *Springer Optimization and Its Applications*. Springer, third edition, 2021.

[14] Qi Chen, Emma S. Johnson, David E. Bernal, Romeo Valentin, Sunjeev Kale, Johnny Bates, John D. Siirola, and Ignacio E. Grossmann. Pyomo.GDP: an ecosystem for logic based modeling and optimization development. *Optimization and Engineering*, 23(1):607–642, 2022.

[15] Zhi Chen, Melvyn Sim, and Peng Xiong. Robust Stochastic Optimization Made Easy with RSOME. *Management Science*, 66(8):3329–3339, 2020.

[16] Zhi Chen and Peng Xiong. RSOME in Python: An Open-Source Package for Robust Stochastic Optimization Made Easy. *INFORMS Journal on Computing*, 35(4):717–724, 2023.

[17] Yves Colombani, Bob Daniel, and Susanne Heipcke. Mosel: A Modular Environment for Modeling and Solving Optimization Problems. In Josef Kallrath, editor, *Modeling Languages in Mathematical Optimization*, volume 88 of *Applied Optimization*, pages 211–238. Kluwer Academic Publishers, 2004.

[18] Marco Colombo, Andreas Grothey, Jonathan Hogg, Kristian Woodsend, and Jacek Gondzio. A structure-conveying modelling language for mathematical and stochastic programming. *Mathematical Programming Computation*, 1(4):223–247, 2009.

[19] Kevin Cunningham and Linus Schrage. The LINGO Algebraic Modeling Language. In Josef Kallrath, editor, *Modeling Languages in Mathematical Optimization*, volume 88 of *Applied Optimization*, pages 159–171. Kluwer Academic Publishers, 2004.

[20] Steven Diamond and Stephen Boyd. CVXPY: A Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5, 2016.

[21] Ulrich Dorndorf, Stefan Droste, and Thorsten Koch. Using Zimpl for Modeling Production Planning Problems. In Josef Kallrath, editor, *Algebraic Modeling Systems: Modeling and Solving Real World Optimization Problems*, Applied Optimization, pages 145–158. Springer, 2012.

[22] Oscar Dowson and Lea Kapelevich. SDDP.jl: A Julia Package for Stochastic Dual Dynamic Programming. *INFORMS Journal on Computing*, 33(1):27–33, 2021.

[23] Iain Dunning, Joey Huchette, and Miles Lubin. JuMP: A Modeling Language for Mathematical Optimization. *SIAM Review*, 59(2):295–320, 2017.

[24] EGRET: Electrical grid research and engineering tools. Version 0.6.2, `https://github.com/grid-parity-exchange/Egret`, 2026.

[25] E. F. D. Ellison and Gautam Mitra. UIMP: User Interface for Mathematical Programming. *ACM Transactions on Mathematical Software*, 8(3):229–255, 1982.

[26] Robert Fourer. Modeling languages versus matrix generators for linear programming. *ACM Transactions on Mathematical Software*, 9(2):143–183, 1983.

[27] Robert Fourer. Extending a General-Purpose Algebraic Modeling Language to Combinatorial Optimization: A Logic Programming Approach. In David L. Woodruff, editor, *Advances in Computational and Stochastic Optimization, Logic Programming, and Heuristic Search*, Operations Research/Computer Science Interfaces Series, pages 31–74. Kluwer Academic Publishers, 1998.

[28] Robert Fourer. Algebraic Modeling Languages for Optimization. In Saul I. Gass and Michael C. Fu, editors, *Encyclopedia of Operations Research and Management Science*, pages 43–51. Springer, 2013.

[29] Robert Fourer and David M. Gay. Extending an Algebraic Modeling Language to Support Constraint Programming. *INFORMS Journal on Computing*, 14(4):322–344, 2002.

[30] Robert Fourer, David M. Gay, and Brian W. Kernighan. A Modeling Language for Mathematical Programming. *Management Science*, 36(5):519–554, 1990.

[31] Robert Fourer, David M. Gay, and Brian W. Kernighan. *AMPL: A Modeling Language for Mathematical Programming*. Duxbury Press, second edition, 2003.

[32] Robert Fourer, David M. Gay, and Brian W. Kernighan. Design Principles and New Developments in the AMPL Modeling Language. In Josef Kallrath, editor, *Modeling Languages in Mathematical Optimization*, volume 88 of *Applied Optimization*, pages 105–135. Kluwer Academic Publishers, 2004.

[33] Alan M. Frisch, Warwick Harvey, Chris Jefferson, Bernadette Martínez-Hernández, and Ian Miguel. Essence: A constraint language for specifying combinatorial problems. *Constraints*, 13(3):268–306, 2008.

[34] Anqi Fu, Balasubramanian Narasimhan, and Stephen Boyd. CVXR: An R Package for Disciplined Convex Optimization. *Journal of Statistical Software*, 94(14), 2020.

[35] Arthur M. Geoffrion. An Introduction to Structured Modeling. *Management Science*, 33(5):547–588, 1987.

[36] Joel Goh and Melvyn Sim. Robust Optimization Made Easy with ROME. *Operations Research*, 59(4):973–985, 2011.

[37] Michael Grant, Stephen Boyd, and Yinyu Ye. Disciplined Convex Programming. In Leo Liberti and Nelson Maculan, editors, *Global Optimization: From Theory to Implementation*, volume 84 of *Nonconvex Optimization and Its Applications*, pages 155–210. Springer, 2006.

[38] Michael C. Grant and Stephen P. Boyd. Graph Implementations for Nonsmooth Convex Programs. In Vincent D. Blondel, Stephen P. Boyd, and Hidenori Kimura, editors, *Recent Advances in Learning and Control*, volume 371 of *Lecture Notes in Control and Information Sciences*, pages 95–110. Springer, 2008.

[39] Harvey J. Greenberg. MODLER: Modeling by object-driven linear elemental relations. *Annals of Operations Research*, 38(1):239–280, 1992.

[40] Harvey J. Greenberg and Frederic H. Murphy. A comparison of mathematical programming modeling systems. *Annals of Operations Research*, 38(1):177–238, 1992.

[41] William E. Hart, Jean-Paul Watson, and David L. Woodruff. Pyomo: modeling and solving mathematical programs in Python. *Mathematical Programming Computation*, 3(3):219–260, 2011.

[42] Hassan Hijazi, Guanglei Wang, and Carleton Coffrin. Gravity: A mathematical modeling language for optimization and machine learning. In *Machine Learning Open Source Software Workshop at NeurIPS 2018*, 2018.

[43] Fabian Hofmann. Linopy: Linear optimization with n-dimensional labeled variables. *Journal of Open Source Software*, 8(84):4823, 2023.

[44] Kenneth Holmström and Marcus M. Edvall. The TOMLAB Optimization Environment. In Josef Kallrath, editor, *Modeling Languages in Mathematical Optimization*, volume 88 of *Applied Optimization*, pages 369–376. Kluwer Academic Publishers, 2004.

[45] Tim Helge Hultberg. FLOPC++ An Algebraic Modeling Language Embedded in C++. In *Operations Research Proceedings 2006*, pages 187–190. Springer, 2007.

[46] T. Hürlimann and J. Kohlas. LPL: A structured language for linear programming modeling. *OR Spektrum*, 10(1):55–63, 1988.

[47] Tony Hürlimann. The LPL Modeling Language. In Josef Kallrath, editor, *Modeling Languages in Mathematical Optimization*, volume 88 of *Applied Optimization*, pages 173–183. Kluwer Academic Publishers, 2004.

[48] IEEE PES Task Force on Benchmarks for Validation of Emerging Power System Algorithms. Power grid lib – unit commitment. `https://github.com/power-grid-lib/pglib-uc`, 2026.

[49] Jordan Jalving, Sungho Shin, and Victor M. Zavala. A graph-based modeling abstraction for optimization: concepts and implementation in Plasmo.jl. *Mathematical Programming Computation*, 14(4):699–747, 2022.

[50] Kristian Jensen, João G. R. Cardoso, and Nikolaus Sonnenschein. Optlang: An algebraic modeling language for mathematical optimization. *The Journal of Open Source Software*, 2(9):139, 2017.

[51] Josef Kallrath. Mathematical Optimization and the Role of Modeling Languages. In Josef Kallrath, editor, *Modeling Languages in Mathematical Optimization*, volume 88 of *Applied Optimization*, pages 3–24. Kluwer Academic Publishers, 2004.

[52] Josef Kallrath, editor. *Algebraic Modeling Systems: Modeling and Solving Real World Optimization Problems*. Applied Optimization. Springer, 2012.

[53] Bernard Knueven, David Mildebrath, Christopher Muir, John D. Siirola, Jean-Paul Watson, and David L. Woodruff. A parallel hub-and-spoke system for large-scale scenario-based optimization under uncertainty. *Mathematical Programming Computation*, 15(4):591–619, 2023.

[54] Bernard Knueven, James Ostrowski, and Jean-Paul Watson. On Mixed Integer Programming Formulations for the Unit Commitment Problem. Optimization Online, `https://optimization-online.org/wp-content/uploads/2018/11/6930.pdf`, 2018.

[55] Bernard Knueven, James Ostrowski, and Jean-Paul Watson. On Mixed-Integer Programming Formulations for the Unit Commitment Problem. *INFORMS Journal on Computing*, 32(4):857–876, 2020.

[56] Thorsten Koch. *Rapid Mathematical Programming*. PhD thesis, Technische Universität Berlin, 2004. ZIB-Report 04-58.

[57] Bjarni Kristjansson and Denise Lee. The MPL Modeling System. In Josef Kallrath, editor, *Modeling Languages in Mathematical Optimization*, volume 88 of *Applied Optimization*, pages 239–266. Kluwer Academic Publishers, 2004.

[58] C.A.C. Kuip. Algebraic languages for mathematical programming. *European Journal of Operational Research*, 67(1):25–51, 1993.

[59] Benoît Legat, Oscar Dowson, Joaquim Dias Garcia, and Miles Lubin. MathOptInterface: A Data Structure for Mathematical Optimization Problems. *INFORMS Journal on Computing*, 34(2):672–689, 2022.

[60] Johan Löfberg. YALMIP: a toolbox for modeling and optimization in MATLAB. In *Proceedings of the 2004 IEEE International Conference on Computer Aided Control Systems Design*, pages 284–289. IEEE, 2004.

[61] Johan Löfberg. Automatic robust convex programming. *Optimization Methods and Software*, 27(1):115–129, 2012.

[62] Miles Lubin, Oscar Dowson, Joaquim Dias Garcia, Joey Huchette, Benoît Legat, and Juan Pablo Vielma. JuMP 1.0: recent improvements to a modeling language for mathematical optimization. *Mathematical Programming Computation*, 15(3):581–589, 2023.

[63] Miles Lubin and Iain Dunning. Computing in Operations Research Using Julia. *INFORMS Journal on Computing*, 27(2):238–248, 2015.

[64] C. Lucas and G. Mitra. Computer-Assisted Mathematical Programming (Modelling) System: CAMPS. *The Computer Journal*, 31(4):364–375, 1988.

[65] Stephen Maher, Matthias Miltenberger, João Pedro Pedroso, Daniel Rehfeldt, Robert Schwarz, and Felipe Serrano. PySCIPOpt: Mathematical Programming in Python with the SCIP Optimization Suite. In *Mathematical Software – ICMS 2016*, volume 9725 of *Lecture Notes in Computer Science*, pages 301–307. Springer, 2016.

[66] Alexander Meeraus. An algebraic approach to modeling. *Journal of Economic Dynamics and Control*, 5(1):81–108, 1983.

[67] Bardhyl Miftari, Mathias Berger, Hatim Djelassi, and Damien Ernst. GBOML: Graph-Based Optimization Modeling Language. *Journal of Open Source Software*, 7(72):4158, 2022.

[68] Stuart Mitchell, Michael O'Sullivan, and Iain Dunning. PuLP: A linear programming toolkit for Python. Optimization Online, `https://optimization-online.org/2011/09/3178/`, 2011.

[69] Nicholas Nethercote, Peter J. Stuckey, Ralph Becket, Sebastian Brand, Gregory J. Duck, and Guido Tack. MiniZinc: Towards a Standard CP Modelling Language. In *Principles and Practice of Constraint Programming – CP 2007*, volume 4741 of *Lecture Notes in Computer Science*, pages 529–543. Springer, 2007.

[70] Bethany Nicholson, John D. Siirola, Jean-Paul Watson, Victor M. Zavala, and Lorenz T. Biegler. pyomo.dae: a modeling and automatic discretization framework for optimization with differential and algebraic equations. *Mathematical Programming Computation*, 10(2):187–223, 2018.

[71] P.C. Piela, T.G. Epperly, K.M. Westerberg, and A.W. Westerberg. ASCEND: an object-oriented computer environment for modeling and analysis: The modeling language. *Computers & Chemical Engineering*, 15(1):53–72, 1991.

[72] Robert E. Pugh. A language for nonlinear programming problems. *Mathematical Programming*, 2(1):176–206, 1972.

[73] Joshua L. Pulsipher, Weiqi Zhang, Tyler J. Hongisto, and Victor M. Zavala. A unifying modeling abstraction for infinite-dimensional optimization. *Computers & Chemical Engineering*, 156:107567, 2022.

[74] Richard E. Rosenthal. Algebraic modeling languages for optimization. In Saul I. Gass and Carl M. Harris, editors, *Encyclopedia of Operations Research and Management Science*, pages 16–19. Springer, 2001.

[75] Guillaume Sagnol and Maximilian Stahlberg. PICOS: A Python interface to conic optimization solvers. *Journal of Open Source Software*, 7(70):3915, 2022.

[76] Maximilian Schaller, Goran Banjac, Steven Diamond, Akshay Agrawal, Bartolomeo Stellato, and Stephen Boyd. Embedded Code Generation With CVXPY. *IEEE Control Systems Letters*, 6:2653–2658, 2022.

[77] Carl A. Schweiger and Christodoulos A. Floudas. The MINOPT Modeling Language. In Josef Kallrath, editor, *Modeling Languages in Mathematical Optimization*, volume 88 of *Applied Optimization*, pages 185–209. Kluwer Academic Publishers, 2004.

[78] Xinyue Shen, Steven Diamond, Yuantao Gu, and Stephen Boyd. Disciplined convex-concave programming. In *Proceedings of the 55th IEEE Conference on Decision and Control*, pages 1009–1014. IEEE, 2016.

[79] Sungho Shin, Mihai Anitescu, and François Pacaud. Accelerating optimal power flow with GPUs: SIMD abstraction of nonlinear programs and condensed-space interior-point methods. *Electric Power Systems Research*, 236:110651, 2024.

[80] Mike Steglich and Thomas Schleiff. *CMPL: Coliop Mathematical Programming Language*. Technische Hochschule Wildau, 2010.

[81] Stefan Theußl, Florian Schwendinger, and Kurt Hornik. ROI: An Extensible R Optimization Infrastructure. *Journal of Statistical Software*, 94(15), 2020.

[82] Madeleine Udell, Karanveer Mohan, David Zeng, Jenny Hong, Steven Diamond, and Stephen Boyd. Convex Optimization in Julia. In *Proceedings of the First Workshop for High Performance Technical Computing in Dynamic Languages*, pages 18–28. IEEE, 2014.

[83] Christian Valente, Gautam Mitra, Mustapha Sadki, and Robert Fourer. Extending Algebraic Modelling Languages for Stochastic Programming. *INFORMS Journal on Computing*, 21(1):107–122, 2009.

[84] Pascal Van Hentenryck. *The OPL Optimization Programming Language*. MIT Press, 1999.

[85] Pascal Van Hentenryck, Laurent Michel, Frederic Paulin, and Jean-Francois Puget. The OPL Studio Modeling System. In Josef Kallrath, editor, *Modeling Languages in Mathematical Optimization*, volume 88 of *Applied Optimization*, pages 307–350. Kluwer Academic Publishers, 2004.

[86] Jean-Paul Watson, David L. Woodruff, and William E. Hart. PySP: modeling and solving stochastic programs in Python. *Mathematical Programming Computation*, 4(2):109–149, 2012.

[87] James S. Welch. PAM—A Practitioner's Approach to Modeling. *Management Science*, 33(5):610–625, 1987.

[88] Christian Wolf, Achim Koberstein, and Tim Hultberg. Stochastic Extensions to FlopC++. In *Operations Research Proceedings 2010*, pages 333–338. Springer, 2011.

[89] Yue Yang, Chenhui Lin, Luo Xu, and Wenchuan Wu. PyOptInterface: Design and implementation of an efficient modeling language for mathematical optimization. arXiv:2405.10130, 2024.